

Product Specification

KODI

AI Phone Assistant — Android

Version 1.0 | April 2026

1. Introduction

Kodi is an AI-powered Android assistant that performs tasks on a user's phone exactly as a human would — by listening to voice commands, understanding intent, and interacting with the device's apps through Android's accessibility and system APIs. Kodi does not wrap existing assistants. It is a self-contained agent loop.

Kodi has one purpose: to be the best AI phone assistant available. Every design decision, constraint, and engineering trade-off in this document is made in service of that single goal. Features that do not directly improve the assistant experience are explicitly out of scope.

1.1 Design Philosophy

- Voice-first. The microphone is always the primary input; no interaction requires screen touch.
- Action-first. Kodi completes tasks, it does not ask unnecessary clarifying questions.
- Memory-first. Kodi remembers everything the user tells it, indefinitely, and uses that context in every response.
- Minimal surface. No dashboard, no settings screen beyond initial setup, no feature flags. One interaction mode.
- On-device where it matters. Wake word detection and hotword classification run locally. Network is only required for STT, LLM, and search.

1.2 What Kodi Is Not

- Not a launcher, widget, or home screen replacement.
- Not a smart home hub or IoT controller.
- Not a productivity suite (no note-taking, no calendar UI, no task manager).
- Not a chatbot with a text interface.

- Not a multi-user assistant. Kodi is scoped to a single device owner.

Note: If a requested feature does not fall within the defined scope, it is deferred. This document is the authoritative scope boundary.

2. Goals and Non-Goals

2.1 Primary Goals

Goal	Success Criterion
Voice activation without touch	Wake word detected within 500 ms, 95%+ accuracy in a quiet room
Intent understood in natural language	Correct tool selected for 90%+ of commands in user testing
App interactions match human actions	WhatsApp message sent, call placed, or search completed within 8 s of command
Memory persists across sessions	User preferences and named facts recalled correctly 30 days after being stated
Sub-3 s response for common tasks	P90 latency ≤ 3 s for send-message and call commands

2.2 Non-Goals

- Voice cloning or custom wake-word training per user (V1).
- Offline LLM inference on device.
- Multi-language support beyond English (V1).
- Proactive / unprompted notifications.
- Third-party skill or plugin system.
- iOS support.

3. System Architecture

Kodi is composed of two primary components: an Android application and a lightweight cloud backend. The Android app handles audio capture, wake-word detection, Accessibility Service execution, and text-to-speech. The backend handles LLM reasoning, memory, and web search.

3.1 Component Overview

Layer	Component	Technology	Runs On
Voice Capture	Always-on mic listener	Android AudioRecord API	Device
Wake Word	Hotword detector	Porcupine SDK (Picovoice)	Device — on-chip
Speech-to-Text	Audio transcription	OpenAI Whisper API	Cloud
Agent Brain	LLM + tool-calling loop	Claude claude-sonnet-4-6 via Anthropic API	Cloud
Memory	Persistent recall layer	Mem0 (self-hosted, Qdrant backend)	VPS
Web Research	Real-time web search	Brave Search API (LLM Context endpoint)	Cloud
Phone Control	UI interaction engine	Android AccessibilityService	Device
Messaging	Native SMS dispatch	Android SmsManager API	Device
Voice Output	Text-to-speech	Android TTS (fallback: VibeVoice-Realtime)	Device
Backend API	Orchestration server	FastAPI + Python 3.12	VPS (Hetzner CX21)

3.2 Request Lifecycle

Every user interaction follows this sequence without exception:

- User speaks. AudioRecord captures 16 kHz PCM frames continuously in a foreground Service.
- Porcupine detects the wake word "Kodi" on-device. Zero network calls at this stage.
- Kodi begins recording the command. VAD (voice activity detection) determines when speech ends.
- Audio is transmitted to OpenAI Whisper API. Transcription returned as plain text.
- Backend receives the transcript. Mem0 is queried for relevant memories. Both are injected into the system prompt.
- Claude processes the prompt with the defined tool schema. It returns one or more tool calls.
- Backend executes tool calls (search, send command to device, etc.) and returns results to Claude.
- Claude produces a final natural-language response.
- Response is sent to the Android app as a text stream. TTS engine speaks it sentence-by-sentence.

- Interaction is written to memory via Mem0 asynchronously (non-blocking).

3.3 Backend Infrastructure

Resource	Specification	Monthly Cost
VPS	Hetzner CX21 — 2 vCPU, 4 GB RAM, 40 GB SSD, Ubuntu 24.04	~\$6
Vector DB	Qdrant (Docker, same VPS)	Included
Memory layer	Mem0 open-source (Docker, same VPS)	Free
LLM	Claude claude-sonnet-4-6 — pay-per-token	~\$10–20
STT	OpenAI Whisper API — \$0.006/min	~\$2–5
Search	Brave Search API — \$5/1k requests, \$5 credit/mo	~\$0–5
Total		~\$18–36/month

4. Android Application

4.1 Technical Requirements

Requirement	Specification
Minimum Android version	API 26 (Android 8.0 Oreo)
Target SDK	API 35 (Android 15)
Language	Kotlin 2.0 + Jetpack Compose
Architecture	MVVM + Repository pattern
Network	Retrofit 2 + OkHttp 4 for backend; separate HttpClient for streaming TTS
Audio	AudioRecord at 16 kHz, mono, PCM 16-bit
Background execution	Foreground Service with microphone type; persistent notification

4.2 Required Permissions

Permission	Purpose	Declared How
RECORD_AUDIO	Continuous mic capture for wake word and commands	Manifest + runtime request
FOREGROUND_SERVICE	Keep assistant alive when screen is off	Manifest
FOREGROUND_SERVICE_MICROPHONE	API 34+ microphone foreground type	Manifest
SEND_SMS	Dispatch SMS natively via SmsManager	Manifest + runtime request
CALL_PHONE	Initiate phone calls programmatically	Manifest + runtime request
READ_CONTACTS	Resolve contact names to numbers	Manifest + runtime request
INTERNET	Backend API communication	Manifest (normal, auto-granted)
BIND_ACCESSIBILITY_SERVICE	Interact with WhatsApp and other apps via UI tree	Manifest (user enables in Settings)

4.3 Wake Word Detection

Porcupine (Picovoice) runs entirely on-device. The model is bundled in the APK at 1–4 MB. Activation phrase is "Hey Kodi". No audio data leaves the device until the wake word is detected.

Parameter	Value
Model	Porcupine built-in "JARVIS" keyword, re-labeled to "Hey Kodi" in V1; custom model in V2
Frame size	512 samples at 16 kHz (32 ms per frame)
False accept rate	< 1 per 10 hours of ambient audio (Picovoice default)
Latency	< 100 ms from end of phrase to detection
CPU overhead	< 2% on ARM Cortex-A55

4.4 Accessibility Service

The AccessibilityService is the core mechanism by which Kodi interacts with third-party apps. It reads the live UI element tree and dispatches actions — taps, text input, scrolls — exactly as a human finger would.

4.4.1 Supported App Interactions (V1)

App / System	Actions Supported
WhatsApp	Open conversation by contact name, type and send message, read last received message
Phone dialer	Dial a number or contact name, end call
SMS (default app)	Open thread, compose and send message (fallback to SmsManager for new threads)
Chrome / browser	Open URL, read page title and visible text
System settings	Toggle Wi-Fi, Bluetooth, airplane mode, brightness, volume
Any app	Launch by package name, navigate back, scroll up/down, tap labeled element

4.4.2 Interaction Flow — WhatsApp Message

- Agent returns tool call: `send_whatapp_message(contact="Mom", message="I'll be home by 7")`.
- AccessibilityService launches WhatsApp via Intent.
- Service polls UI tree until search bar is visible (max 3 s timeout).
- Tap search bar → type contact name → wait for result list → tap first match.
- Wait for conversation view → tap message field → type message → tap Send button.
- Confirm send: verify sent message appears in UI tree. Report success or failure to agent.
- Return to previous app state using `GLOBAL_ACTION_HOME` or `GLOBAL_ACTION_BACK`.

4.4.3 Play Store Policy Compliance

Kodi's AccessibilityService is scoped to automation with explicit user consent. The service metadata declares automation purpose (not `isAccessibilityTool`). Before enabling, a required disclosure screen explains exactly what Kodi will access. All actions require a prior voice command — no autonomous or unprompted actions are performed. This complies with Google Play Developer Policy Section 4.8 (Accessibility API).

5. Agent and Tool System

5.1 LLM Configuration

Parameter	Value
Model	claude-sonnet-4-6 (Anthropic)
Max tokens (output)	1,024 per turn
Temperature	0.3 (low, for deterministic tool use)
System prompt	Fixed 400-token system prompt injected at every turn (see Section 5.2)
Memory injection	Top-5 Mem0 results prepended to user message, max 800 tokens
Tool choice	auto — model selects tools based on intent

5.2 System Prompt (Canonical)

The following system prompt is injected verbatim at every request. It may not be modified by users or runtime configuration in V1.

```
You are Kodi, a voice AI assistant running on the user's Android phone. You execute tasks on the phone exactly as a human would, using the provided tools. Be brief. Confirm what you did, not what you're about to do. When a task requires a tool, call it immediately – do not ask for confirmation unless the action is irreversible and ambiguous. Never refuse a task within your tool set. Never explain your limitations unless asked.
```

5.3 Tool Schema (V1)

Tool Name	Description	Key Parameters
send_whatsapp_message	Send a message via WhatsApp	contact: str, message: str
send_sms	Send an SMS via Android SmsManager	contact: str, message: str
make_phone_call	Dial a contact or number	contact: str
search_web	Search the web and return summarised context	query: str, deep: bool = false
open_app	Launch an app by name or package	app_name: str
toggle_setting	Change a system setting	setting: str, value: bool int
remember	Store a fact in long-term memory	content: str
recall	Retrieve facts from memory	query: str

Tool Name	Description	Key Parameters
get_contacts	Look up a contact's phone number	name: str
read_last_message	Read the most recent WhatsApp or SMS message from a contact	contact: str, app: str

Note: No additional tools are added in V1 without a specification amendment. Tool sprawl is the most common cause of degraded agent reliability.

5.4 Tool Execution Rules

- All tools time out at 10 seconds. On timeout, the agent reports failure and stops.
- Tool calls are executed sequentially in V1. Parallel execution is a V2 consideration.
- If a tool fails, the agent reports what failed and why, in one sentence. It does not retry automatically.
- The agent may chain at most 4 tool calls per user command. Beyond 4, it reports partial completion.

6. Memory System

6.1 Architecture

Kodi uses Mem0 (open-source, self-hosted) with Qdrant as the vector store. Both run in Docker on the same VPS as the FastAPI backend. Memory is stored per user (single user in V1, identified by device ID).

6.2 Memory Types

Type	Examples	TTL
User facts	"My name is Thabo." "I live in Lusaka." "I prefer voice calls over texts."	Permanent
Preferences	"Always confirm before calling." "Use formal language with my boss."	Permanent
Episodic	"Yesterday Kodi sent a message to Mom about dinner."	90 days
Contextual	"User is currently in a meeting." (inferred from Do Not Disturb state)	Session only

6.3 Memory Lifecycle

- **Write:** after every completed interaction, Mem0 extracts and stores salient facts asynchronously. This does not block the response.
- **Read:** before every LLM call, the user's message is used to query Mem0 (top-5 results by semantic similarity). Results are injected into the prompt.
- **Forget:** user can say "Kodi, forget that" to delete the last written memory. "Kodi, forget everything" triggers full memory wipe with confirmation.
- **Privacy:** all memory data stays on the user's self-hosted VPS. Nothing is sent to third parties.

7. Voice Input / Output

7.1 Speech-to-Text

Parameter	Value
Provider	OpenAI Whisper API (whisper-1 model)
Format sent	WAV, 16 kHz, mono, 16-bit PCM
Max recording duration	30 seconds. Longer commands are chunked.
Cost	\$0.006 per minute of audio
Expected latency	300–800 ms for a 5-second command
Fallback	Android SpeechRecognizer API (on-device, no cost, lower accuracy)

7.2 Text-to-Speech

Parameter	Value
Primary engine	Android TextToSpeech API (built-in, zero cost, zero latency)
Voice	en-US-Neural (default Google TTS voice on device)
Secondary engine	VoiceVoice-Realtime-0.5B (self-hosted, English only, requires GPU node — V2)
Streaming	Response is spoken sentence-by-sentence as it streams from the backend
Interruption	User speaking while TTS is active cancels current speech immediately

8. Web Research Tool

8.1 Implementation

Web research is implemented as a two-step pipeline: search → extract. The LLM receives structured context, not raw HTML.

Step	Component	Details
1. Search	Brave Search API — /res/v1/llm/context	Returns pre-chunked, model-ready text. Avoids raw HTML parsing.
2. Extract (deep mode)	Jina Reader (r.jina.ai prefix)	Free, keyless, returns clean markdown from any URL. Used for full-page reads.
3. Cache	SQLite on VPS	Identical queries within 5 minutes are served from cache. Zero additional API cost.

8.2 Search Modes

- Fast (default): One Brave LLM Context call. Returns in < 1.5 s. Used for current facts, scores, prices, news.
- Deep (explicit): Brave search + top-3 page fetches via Jina. Used when user says "research", "look into", or "tell me about". Returns in < 6 s.

8.3 Cost Control

- Brave API: \$5 in monthly credits covers ~1,000 queries. A personal assistant at 20 searches/day uses ~600/month — within the free credit.
- Cache hit rate target: > 40% for weather, sports, and repeated factual queries.
- Deep mode is only triggered by explicit user intent signal. It cannot be triggered by the LLM autonomously.

9. Security and Privacy

9.1 Data Flows

Data Type	Where It Goes	Stored?
Wake word audio	On-device Porcupine model only	Never stored
Command audio	OpenAI Whisper API (HTTPS)	Not stored per OpenAI Zero Data Retention policy
Transcribed text	Backend VPS → Anthropic API	Stored in VPS logs (configurable retention)

Data Type	Where It Goes	Stored?
Memory facts	VPS Qdrant instance only	Stored on VPS indefinitely until deleted by user
Search queries	Brave Search API	Not stored per Brave privacy policy
Contact names/numbers	Resolved on-device only	Never leaves device

9.2 Security Requirements

- All backend API calls use HTTPS with certificate pinning in the Android app.
- Backend API authenticated via device-bound HMAC token generated at first setup. Token is stored in Android Keystore.
- No user credentials are stored in plaintext anywhere in the system.
- Backend runs as a non-root systemd service. SSH key authentication only; password login disabled.
- VPS firewall: only ports 443 (HTTPS) and 22 (SSH) exposed publicly.

9.3 Accessibility Service Boundaries

- The service only performs actions when triggered by a user voice command in the current session.
- The service does not read screen content proactively or log UI state between commands.
- Banking apps, password managers, and apps marked sensitive are excluded from UI tree reading.

10. Error Handling and Degradation

Failure Mode	Kodi's Behaviour
Wake word missed	Silent. User repeats. No error state.
No internet at STT	Falls back to Android on-device SpeechRecognizer. Notifies user once.
Backend unreachable	Speaks: "I'm having trouble connecting. Try again in a moment."
LLM timeout (> 15 s)	Speaks: "That took too long. Please try again."
Target app not installed	Speaks: "I can't find WhatsApp on this phone." Stops.
Accessibility action fails	Retries once. If still fails, reports: "I wasn't able to complete that in [app name]."

Failure Mode	Kodi's Behaviour
Contact not found	Asks once: "I don't recognise [name]. Can you check the spelling?"
Memory write fails	Silent. Logged server-side. Does not interrupt the conversation.

11. Performance Requirements

Metric	Target	Measurement Method
Wake word latency	< 500 ms	Time from end of phrase to visual indicator
STT turnaround	< 1 s for 5-second command	Backend log: STT request → response
Simple task completion (send SMS)	< 4 s end-to-end	Time from command end to TTS confirmation start
Web search response (fast mode)	< 3 s	Backend log: tool call → result
Web search response (deep mode)	< 8 s	Backend log: tool call → result
Memory query latency	< 200 ms	Qdrant query response time in Mem0 logs
Battery drain (always-on mic)	< 3% per hour additional drain	Android battery historian, device idle
App memory footprint	< 80 MB RSS	Android Memory Profiler, foreground state

12. Setup and Onboarding

First-run onboarding is completed in under 5 minutes with three steps. There is no account creation, no email, no cloud subscription sign-up beyond API keys.

12.1 Onboarding Steps

- Step 1 — Permissions: App requests microphone, contacts, phone, and SMS permissions one at a time, with a plain-language explanation for each.
- Step 2 — Backend URL: User enters the HTTPS URL of their VPS backend. App tests connectivity and shows a green/red indicator.

- Step 3 — Accessibility Service: App opens the Android Settings deep-link for Accessibility → Kodi. User enables the toggle. App confirms and proceeds.

After these three steps, Kodi speaks: "I'm ready. Say Hey Kodi to get started." The app has no further configuration screens.

13. Explicitly Out of Scope (V1)

The following are deferred to future versions or permanently excluded. They are listed here to prevent scope creep during development.

Feature	Status	Reason
Multi-language STT / TTS	V2	VibeVoice-Realtime supports English only; Whisper supports all but TTS alignment is complex
Custom wake word training	V2	Requires Porcupine Trainer; adds onboarding complexity
On-device LLM	V3	Requires 6+ GB RAM and GPU on device; not viable for mainstream Android in 2026
Proactive reminders / notifications	Excluded	Requires always-on reasoning loop; increases cost and complexity nonlinearly
WhatsApp group messaging	V2	Group UI tree navigation is fragile and version-specific
iOS support	Excluded	SiriKit restrictions prevent the necessary level of system access
Third-party plugin system	Excluded	Contradicts the minimal-surface principle; increases attack surface
Smart home / IoT control	Excluded	Out of core scope; requires per-device protocol integrations
Text-based chat UI	Excluded	Kodi is a voice assistant. Adding a text UI splits design attention.
Multi-user / family mode	V3	Speaker identification and per-user memory isolation require voice fingerprinting

14. Versioning Roadmap

Version	Milestone	Key Additions
V1.0	Initial release	All features in this specification
V1.1	Stability	Improved Accessibility retry logic, expanded app compatibility list, latency profiling
V2.0	Language + multi-app	Multi-language STT, custom wake word, WhatsApp group messaging, parallel tool execution
V3.0	On-device	On-device LLM inference (Gemma 3B), multi-user memory isolation, speaker recognition

15. Glossary

Term	Definition
AccessibilityService	Android framework API that provides read/write access to the UI element tree of all apps on the device
LLM	Large Language Model — the AI model (Claude) that interprets intent and selects tools
Mem0	Open-source memory layer that extracts, stores, and retrieves facts from conversations using a vector database
Porcupine	On-device wake-word detection SDK by Picovoice; runs without a network connection
Qdrant	Open-source vector database used by Mem0 to store and semantically search memory embeddings
STT	Speech-to-Text — the process of converting spoken audio to a text transcript
TTS	Text-to-Speech — the process of converting a text response to spoken audio
Tool calling	Mechanism by which the LLM selects and invokes a structured function (e.g. <code>send_sms</code>) instead of responding with plain text
UI tree	The structured representation of all visible interface elements on screen, exposed by the Android AccessibilityService
VAD	Voice Activity Detection — algorithm that determines when a user has finished speaking
VPS	Virtual Private Server — the cloud compute instance (Hetzner CX21) hosting the Kodi backend

Term	Definition
Whisper	OpenAI's speech recognition model, used for transcribing user voice commands